Karachi Institute of Economics & Technology
College of Computing & Information Sciences

Software Construction & Development

Lab Manual

Compiled by: Muhammad Rayan Shaikh

Karachi Institute of Economics & Technology
College of Computing & Information Sciences

Software Construction & Development

Lab Manual

Compiled by: Muhammad Rayan Shaikh

CID: _____ Semester: _____

Student ID: _____

Student Name: _____

Instructor Name: _____

KARACHI INSTITUTE OF ECONOMICS & TECHNOLOGY

College of Computing & Information Sciences

Software Construction & Development Lab Manual

INDEX

Date	Particulars	Lab Instructor Signature	Course Teacher Signature
	Lab 01:Introduction to Software Development Life Cycle & Project Planning		
	Lab 02:Introduction to UML Daigrams		
	Lab 03:Understanding of Computer Architectures MVC		
	Lab 04:Understanding of Computer Architectures N-Tier		
	Lab 05:Refactoring In Python		
	Lab 06:Implementing Exception Handling and Robustness Using Python		
	Lab 07:Software Testing in python		
	Lab 08:Making an Executable .exe file		
	Lab 9:Introduction to GitHub and Repository Management		
	Lab 10 :Github Version Control		
	Lab 11: Personal and Peer Reviews		
	Lab 12: Open Ended		

Lab 01

Introduction to Software Development Life Cycle & Project Planning

Objective

Understand the **Software Development Life Cycle (SDLC)** and various **software development models**. They will learn how to create **project plans using planning tools.**

Software Development Life Cycle (SDLC) – Detailed Theory

The **Software Development Life Cycle (SDLC)** is a systematic process used for developing software applications. It ensures that the software is built with high quality, within budget, and in a structured manner. The SDLC consists of several **phases**, each contributing to the successful completion of a software project.

Phases of SDLC

1. Requirement Analysis

- The first step in SDLC involves gathering and analyzing the requirements from stakeholders (clients, users, business teams).
- The goal is to understand **what the software should do** and define functional and non-functional requirements.
- Tools: **Interviews, surveys, brainstorming sessions, Software Requirement Specification (SRS) document.**

---

2. Planning

- In this phase, project managers estimate the **time, cost, and resources** required for development.
- A detailed **project roadmap** is created, including deadlines and team assignments.
- Risk assessment and feasibility analysis are conducted to identify potential challenges.

---

3. Design

- The system architecture is designed based on the requirements.
- This includes **UI/UX design**, database structure, technology selection, and overall system flow.
- The design is documented in **Software Design Document (SDD)**, including flowcharts, wireframes, and ER diagrams.

---

4. Implementation (Coding/Development)

- This is the phase where developers start **writing the actual code** based on the design.
- The system is developed in modules, which are later integrated.
- Programming languages, frameworks, and tools are chosen based on project needs (e.g., Python, Java, React, MySQL).

---

5. Testing

- In this phase, testers check the software for **bugs, security vulnerabilities, and performance issues**.
- Different types of testing include:
  - **Unit Testing:** Checking individual components.
  - **Integration Testing:** Ensuring different modules work together.
  - **System Testing:** Checking the entire system.
  - **User Acceptance Testing (UAT):** Validating if the software meets business needs.

---

6. Deployment

- After successful testing, the software is **deployed** for users.
- Deployment can be **phased (gradual rollout)** or **full release**.
- Cloud platforms like AWS, Google Cloud, or on-premises servers are used.

---

7. Maintenance

- Even after deployment, software requires **regular updates and bug fixes** to ensure smooth operation.
- New features may be added based on user feedback.
- Security patches and performance optimizations are regularly implemented.

Benefits of SDLC

Structured Approach: Ensures systematic development.
Better Quality: Reduces bugs and improves software reliability.
Cost-Efficient: Helps avoid unnecessary expenses.
Predictability: Clear timelines and resource allocation.

Software Development Models – Detailed Explanation

Software development models provide structured methodologies for building software applications efficiently. Each project has unique requirements, so selecting the right model ensures successful execution. Here are some commonly used software development models:

A. Waterfall Model

Definition:

The **Waterfall Model** is a **linear and sequential** approach to software development, where each phase must be completed before moving to the next. It follows a top-down approach and does not allow going back to previous phases.

Phases of the Waterfall Model:

1. **Requirement Analysis:** Gathering all necessary requirements before starting development.
2. **System Design:** Creating architectural and technical design documents.
3. **Implementation (Coding):** Writing the actual program code based on the design.
4. **Testing:** Identifying and fixing bugs after coding is completed.
5. **Deployment:** Releasing the software for users.
6. **Maintenance:** Updating the software and fixing post-deployment issues.

Advantages of the Waterfall Model:

✓ **Simple and Easy to Understand:** The step-by-step approach makes it easy to follow.
✓ **Structured Process:** Each phase has well-defined goals and deliverables.
✓ **Works Well for Small Projects:** Suitable for projects where requirements are clearly defined and unlikely to change.

Disadvantages of the Waterfall Model:

❌ **Rigid Structure:** Once a phase is completed, returning to a previous stage is difficult.
❌ **Late Discovery of Issues:** Testing happens at the end, making it costly to fix major bugs.
❌ **Not Suitable for Changing Requirements:** If customer needs change, modifications are difficult.

When to Use the Waterfall Model?

- When the project requirements are **fixed and well-documented**.
- When the **technology is stable** and not expected to change.

- Ideal for **government projects, payroll systems, and banking software** where all functionalities are predefined.

💡 **Example:** A **Library Management System**, where book cataloging, member registration, and borrowing rules are fixed.

B. Agile Model

Definition:

The **Agile Model** is an **iterative and incremental** software development approach. Instead of developing the entire system at once, it is built in **small cycles (sprints)** with continuous feedback and improvements. Agile focuses on delivering a working product quickly while adapting to changes throughout development.

Phases of the Agile Model:

1. **Concept & Planning:** Identify core features and business goals.
2. **Iteration (Sprint) Planning:** Break the project into smaller tasks (sprints).
3. **Design & Development:** Develop and test a small feature in each sprint.
4. **Review & Feedback:** Gather feedback from users after each sprint.
5. **Deployment & Continuous Improvement:** Release working features frequently and update them based on feedback.

Advantages of the Agile Model:

✓ **More Flexible & Adaptive:** Changes can be made at any stage.
✓ **Faster Delivery:** A working version of the software is available after each sprint.
✓ **Continuous Client Involvement:** Clients provide regular feedback to improve the product.

Disadvantages of the Agile Model:

❌ **High Team Involvement Needed:** Developers and clients must collaborate constantly.
❌ **Less Detailed Documentation:** Agile focuses on working software rather than documentation.
❌ **Difficult to Predict Cost & Timeline:** Since changes are frequent, budgeting can be challenging.

When to Use the Agile Model?

- When requirements **change frequently** based on user needs.
- Best for **startups, mobile applications, and web development projects**.
- Suitable for **e-commerce platforms and social media apps** that need constant updates.

💡 **Example: Instagram**, where new features (stories, reels, filters) are frequently added based on user feedback.

C. Spiral Model

Definition:

The **Spiral Model** is a **risk-driven software development model** that combines iterative development with systematic risk management. It follows a **cyclical approach**, meaning the project passes through multiple phases repeatedly, refining the software at each stage.

Phases of the Spiral Model:

1. **Planning:** Identify objectives, alternatives, and constraints.
2. **Risk Analysis:** Evaluate technical risks and challenges before development.
3. **Development & Prototyping:** Develop a prototype, test it, and refine it in multiple cycles.
4. **Evaluation & Review:** Gather feedback, improve the system, and repeat the cycle until completion.

Advantages of the Spiral Model:

✓ **Risk Management:** Identifies and reduces risks early in development.
✓ **Good for Large & Complex Projects:** Allows for controlled and gradual development.
✓ **Early Identification of Issues:** Problems are addressed before they become major.

Disadvantages of the Spiral Model:

❌ **Expensive & Time-Consuming:** Requires more resources and time compared to other models.
❌ **Requires Skilled Developers:** Risk assessment needs experienced professionals.
❌ **Not Suitable for Small Projects:** The complexity of this model makes it impractical for smaller applications.

When to Use the Spiral Model?

- When developing software with **high security and accuracy requirements**.
- Best for **large-scale enterprise applications, military systems, and AI-based solutions**.
- Suitable for **financial applications, healthcare software, and space research projects**.

💡 **Example: AI-Based Medical Diagnosis System**, where accuracy and data security are critical. Since risks (false diagnoses, data breaches) are high, the **Spiral Model** ensures careful testing and improvements at every stage.

Comparison of Software Development Models

Feature	Waterfall Model	Agile Model	Spiral Model
Approach	Linear & Sequential	Iterative & Incremental	Cyclical & Risk-Driven
Flexibility	Low	High	Medium
Risk Management	Low	Medium	High
Client Involvement	Low	High	Medium
Best for	Fixed projects	Rapidly changing projects	Large and high-risk projects
Example	Library System	Instagram	AI Medical System

What is Project Planning?

Project planning is the process of defining the scope, objectives, tasks, resources, timeline, and budget of a project to ensure its successful completion. It serves as a roadmap that guides teams from project initiation to completion.

Key Components of Project Planning:

1. **Project Scope:** Defines the goals, deliverables, and limitations.
2. **Task Breakdown:** Breaking down the work into manageable activities (Work Breakdown Structure - WBS).
3. **Resource Allocation:** Assigning team members, equipment, and materials.
4. **Timeline & Scheduling:** Setting deadlines and dependencies between tasks.
5. **Risk Management:** Identifying potential risks and planning mitigations.
6. **Budgeting:** Estimating project costs and tracking expenses.
7. **Monitoring & Controlling:** Tracking progress and making adjustments when needed.

Introduction to Microsoft Project (MS Project)

Microsoft Project (MS Project) is a **project management software** used to plan, schedule, track, and manage projects. It helps project managers create detailed project plans, assign resources, and monitor progress using Gantt charts, timelines, and reports.

Key Features of MS Project:

✓ **Gantt Charts:** Visual timeline for tracking tasks and dependencies.
✓ **Task Management:** Assign, prioritize, and track tasks easily.
✓ **Resource Management:** Allocate team members, equipment, and costs.
✓ **Budget Tracking:** Monitor costs to stay within budget.
✓ **Risk Management:** Identify bottlenecks and adjust schedules accordingly.
✓ **Reports & Dashboards:** Generate progress reports for stakeholders.

How to Create a Project Plan in MS Project

Step 1: Create a New Project

1. Open **Microsoft Project**.
2. Click on **File → New → Blank Project**.
3. Save the project file using **File → Save As**.

Step 2: Define the Project Start Date

1. Go to **Project → Project Information**.
2. Set the **Start Date** of your project.
3. Choose whether to **schedule tasks from the Start Date or End Date** (for deadline-driven projects).

Step 3: Add Tasks to the Project

1. In the **Task Name** column, type the list of tasks.
2. Press **Enter** after each task.
3. Use **Indent/Outdent** to create a Work Breakdown Structure (WBS).

💡 **Example:** If planning a Software Development Project, tasks can be:
✅ Requirement Gathering
✅ Design Phase
✅ Development Phase
✅ Testing Phase
✅ Deployment
✅ Maintenance

Step 4: Set Task Durations & Dependencies

1. In the **Duration** column, enter the estimated time for each task (e.g., 5 days, 2 weeks).
2. To create dependencies (link tasks):
    - Select two tasks, click **Task → Link Tasks** (or use “Predecessors” column).
    - Types of dependencies:
        - **Finish-to-Start (FS)**: Task B starts only after Task A finishes.
        - **Start-to-Start (SS)**: Task B starts at the same time as Task A.
        - **Finish-to-Finish (FF)**: Task B finishes only after Task A finishes.
        - **Start-to-Finish (SF)**: Task B finishes before Task A starts.

💡 **Example:** In software development, "Testing" depends on "Development" (Finish-to-Start).

Step 5: Assign Resources

1. Go to **View → Resource Sheet**.
2. Add **Resource Name** (e.g., Developer, Designer, Tester).
3. Assign them to tasks in the **Resource Name** column.
4. Define resource availability and cost rates if needed.

Step 6: Create a Gantt Chart

1. Click on **View → Gantt Chart**.
2. Adjust taskbars by dragging them to modify schedules.
3. Use **Critical Path Analysis** (Project → Critical Path) to identify key tasks.

Step 7: Set Project Milestones

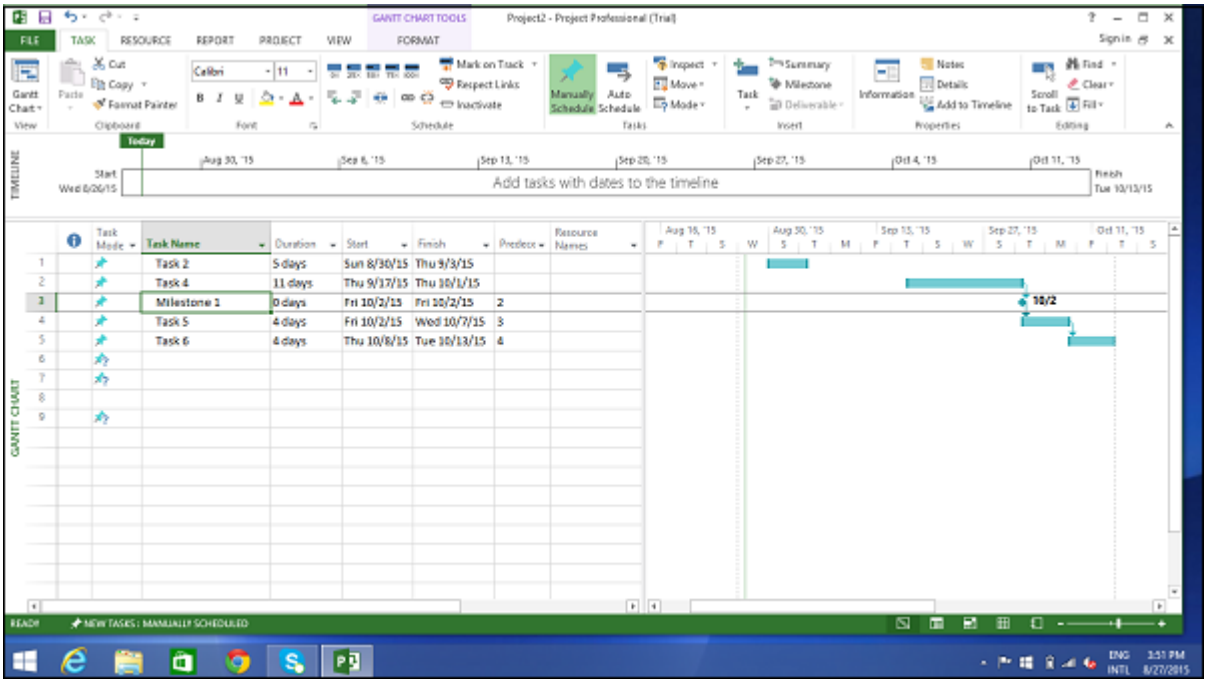
1. Milestones mark significant stages (e.g., completion of a phase).
2. In the **Duration** column, set it to **0 days** (MS Project marks it as a diamond symbol).
3. Example milestones:
    - "Requirement Gathering Completed"
    - "Development Phase Completed"
    - "Final Software Release"

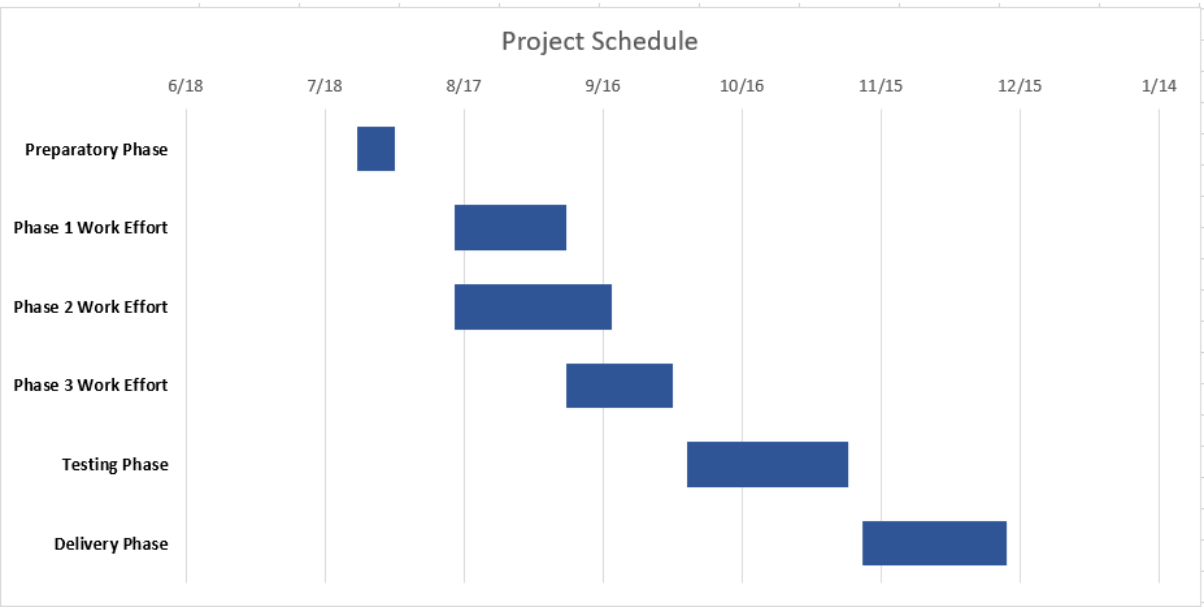
Step 8: Track Progress & Generate Reports

1. Update task completion percentages in the **% Complete** column.
2. View progress in the Gantt Chart.
3. Generate reports using **Report → Dashboard Reports** for stakeholders.

Example: Project Plan for a Software Development Project in MS Project

Task Name	Duration	Start Date	End Date	Predecessor	Resource Assigned
Project Planning	5 days	01-Apr	05-Apr	-	Project Manager
Requirement Gathering	7 days	06-Apr	12-Apr	1	Business Analyst
System Design	10 days	13-Apr	22-Apr	2	Architect
Development Phase	20 days	23-Apr	12-May	3	Developers
Testing Phase	15 days	13-May	27-May	4	QA Engineers
Deployment & Delivery	5 days	28-May	02-Jun	5	DevOps Team


Project2 - Project Professional (Trial)
Add tasks with dates to the timeline
Task Name
Duration
Start
Finish
Task 2 5 days Sun 8/30/15 Thu 9/3/15
Task 4 11 days Thu 9/17/15 Thu 10/1/15
Milestone 1 0 days Fri 10/2/15 Fri 10/2/15 2
Task 5 4 days Fri 10/2/15 Wed 10/7/15 3
Task 6 4 days Thu 10/8/15 Tue 10/13/15 4


Project Schedule
6/18
7/18
8/17
9/16
10/16
11/15
12/15
1/14
Preparatory Phase
Phase 1 Work Effort
Phase 2 Work Effort
Phase 3 Work Effort
Testing Phase
Delivery Phase

Lab 02

Introduction to UML Daigrams

Objective

Will learn how to create **design diagrams** like **UML, Use Case, and Sequence and Components diagrams** using appropriate software

Unified Modeling Language (UML) – Detailed Explanation

1. Introduction to UML

Unified Modeling Language (UML) is a standardized visual modeling language used to design and document software systems. It provides various diagrams to represent different aspects of a system, including its structure, behavior, and interactions. UML is widely used in **software engineering, system architecture, and business process modeling**.

Why Use UML?

✓ **Standardized & Universal** – Used across industries.
✓ **Enhances Communication** – Helps developers, designers, and stakeholders understand system design.
✓ **Improves Software Design** – Provides a clear blueprint before development.
✓ **Easy to Maintain & Update** – Helps document system architecture.

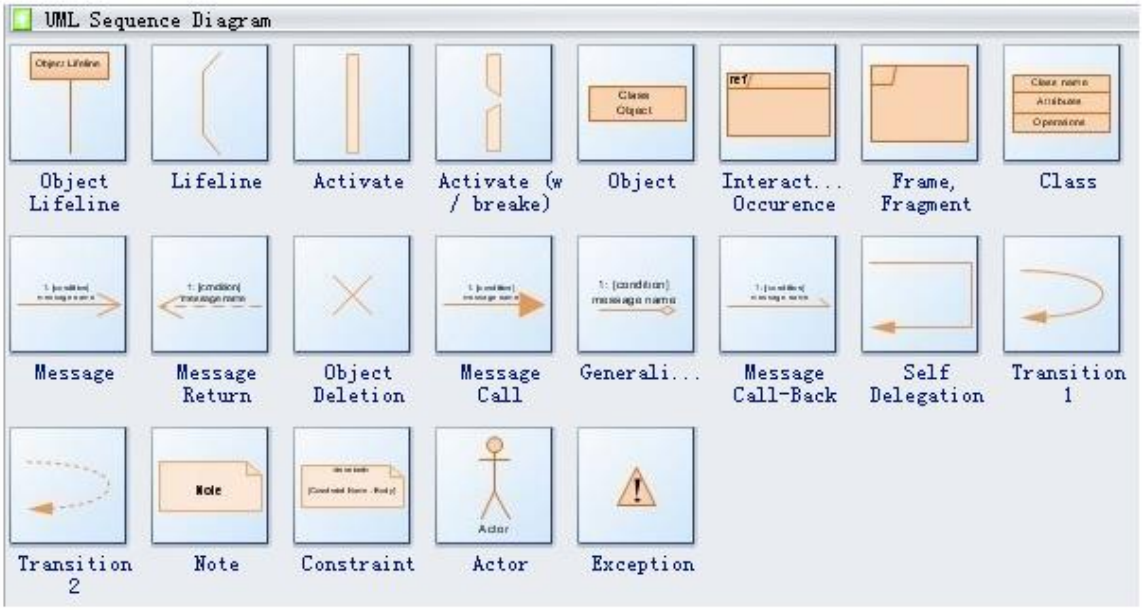
---

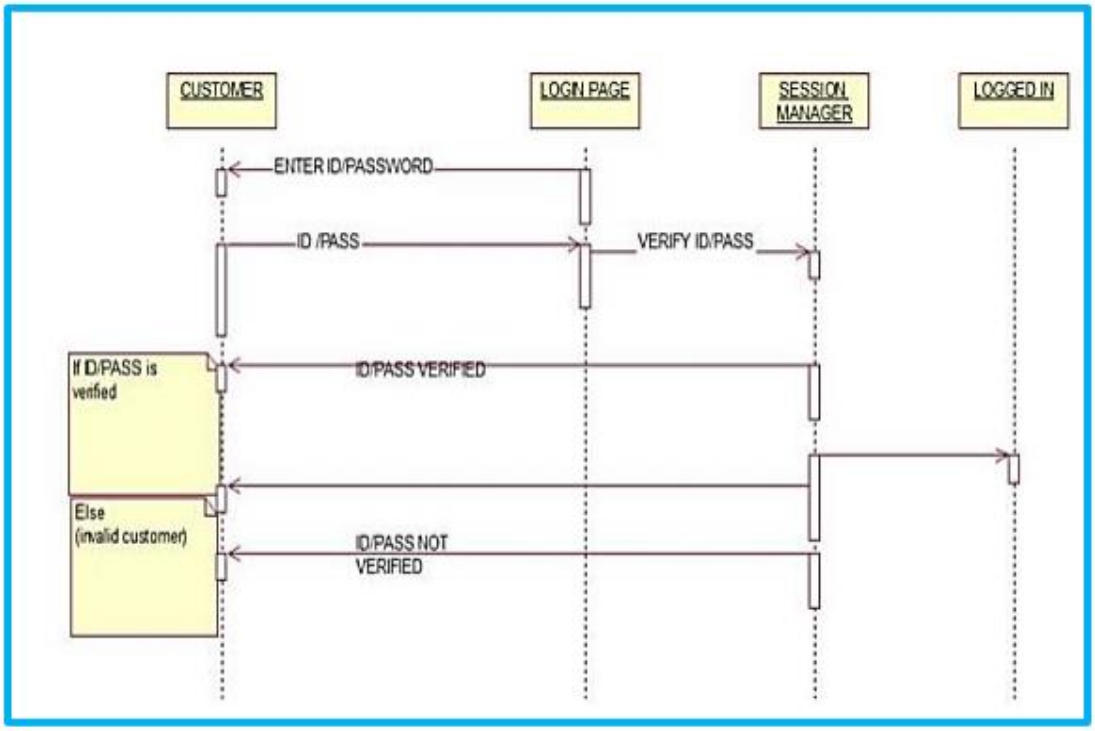
2. UML Sequence Diagram (Behavioral Diagram)

A **Sequence Diagram** represents the interaction between different objects over time. It shows how messages are exchanged between objects to complete a process.

Key Concepts of Sequence Diagram

- **Actors & Objects** – Entities interacting with the system.
- **Lifelines** – Represent the existence of objects over time.
- **Messages** – Arrows showing communication between objects.
- **Activation Bar** – Time duration when an object is active.
- **Loop & Alternative Blocks** – Represent conditional and repetitive interactions.


UML Sequence Diagram
Object Lifeline
Lifeline
Activate
Activate (w / breake)
Object
Interact... Occurence
Frame, Fragment
Class
Message
Message Return
Object Deletion
Message Call
Generali...
Message Call-Back
Self Delegation
Transition 1
Transition 2
Note
Constraint
Actor
Exception


CUSTOMER
LOGIN PAGE
SESSION MANAGER
LOGGED IN
ENTER ID/PASSWORD
ID /PASS
VERIFY ID/PASS
If ID/PASS is verified
ID/PASS VERIFIED
Else (invalid customer)
ID/PASS NOT VERIFIED

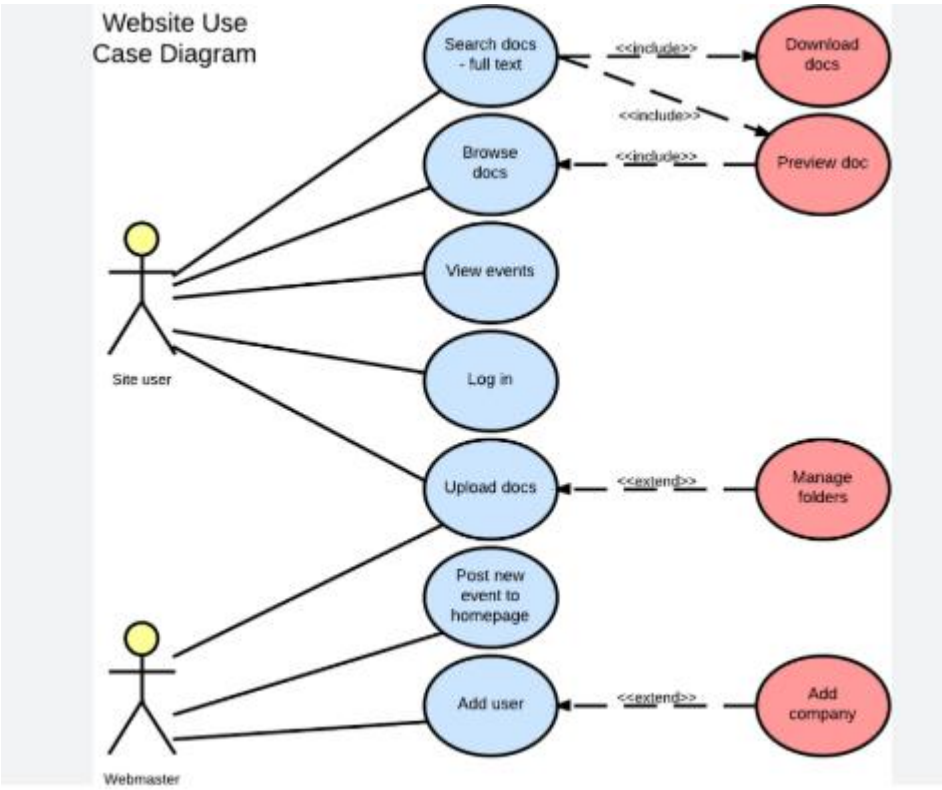
3. UML Use Case Diagram (Behavioral Diagram)

A **Use Case Diagram** represents the functionality of a system by showing interactions between users (actors) and the system. It helps in understanding system behavior from a user's perspective.

Key Concepts of Use Case Diagram

- **Actors** – Users or external systems interacting with the application.
- **Use Cases** – Functionalities provided by the system.
- **Relationships** – Show dependencies between actors and use cases.

Shape	Name
Actor (Stick Figure)	Represents a person or an external system interacting with the software.
Use Case (Oval Shape)	Represents a specific functionality or service provided by the system.
Association (Arrow or Line)	Shows interaction between an actor and a use case.
Include (Dashed Arrow with '<<include>>' label)	Represents mandatory functionality used by another use case.
Extend (Dashed Arrow with '<<extend>>' label)	Represents optional functionality executed under certain conditions.
System Boundary (Rectangle)	Represents the system's scope and defines what is inside or outside.

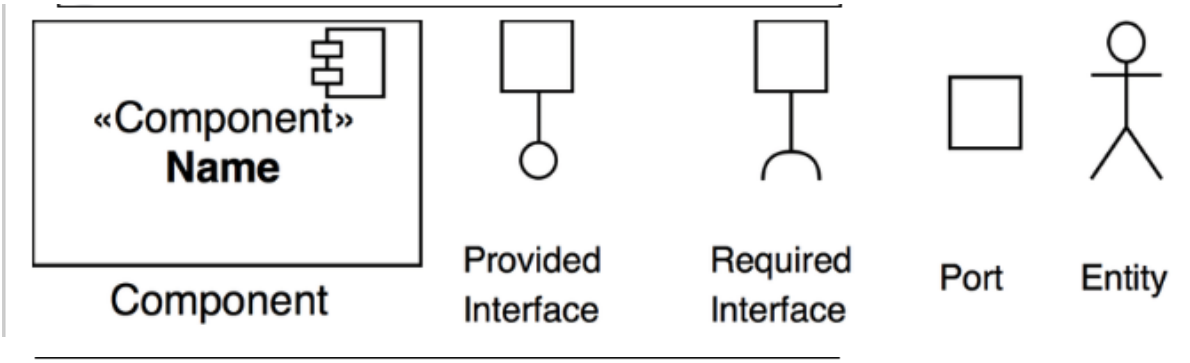


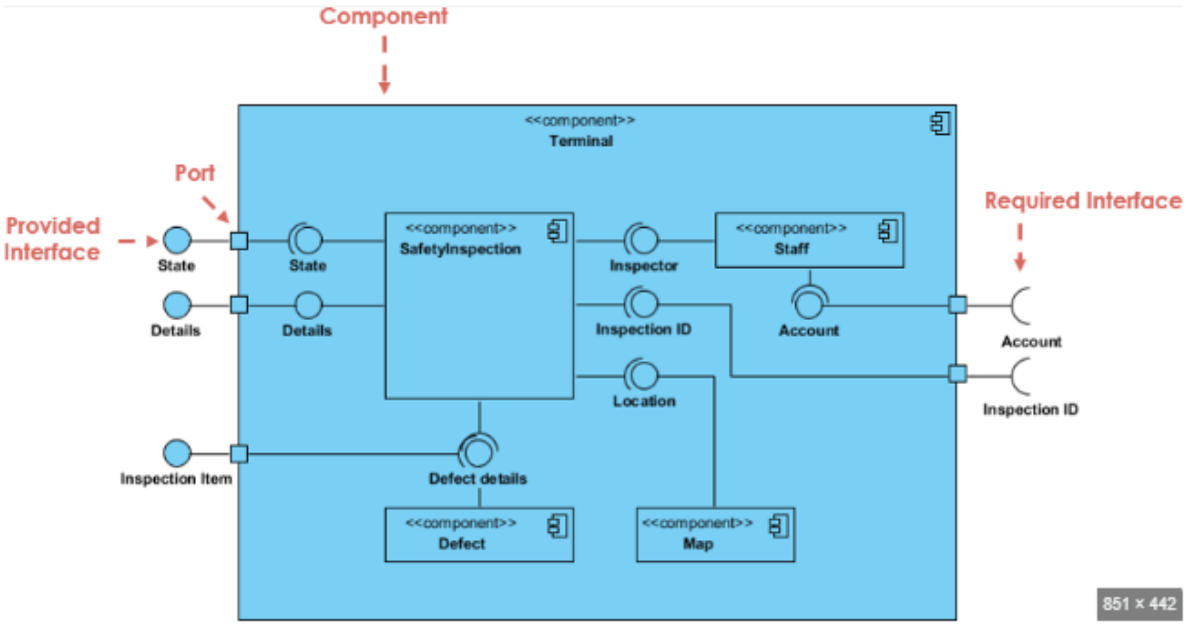
4. UML Component Diagram (Structural Diagram)

A **Component Diagram** represents the physical components of a software system and how they communicate. It helps in understanding **software architecture, dependencies, and modularity**.

Key Concepts of Component Diagram

- **Components** – Independent software modules or executables.
- **Interfaces** – Define how components interact.
- **Dependencies** – Show relationships between components.





---

5. Summary of UML Diagrams

UML Diagram	Purpose	Example Use
Sequence Diagram	Represents interactions between objects over time.	User login system, order processing.
Use Case Diagram	Represents system functionality from a user's perspective.	Online shopping system, ATM transactions.
Component Diagram	Represents software architecture and module dependencies.	Web application, microservices architecture.

---

6. Conclusion

UML diagrams are essential for software design, documentation, and communication. Understanding **Sequence, Use Case, and Component Diagrams** helps in visualizing system interactions, user requirements, and software architecture effectively.

Lab 03

Understanding of Computer Architectures MVC

Objective

Will learn how to Implement MVC architecture

1. What is Software Architecture?

Software Architecture refers to the high-level structuring of a software system. It involves designing the components or modules of a software system and how they interact with one another. Essentially, software architecture defines the blueprint of the software, helping developers make key decisions on how to organize and structure the system.

Key aspects of software architecture include:

- **Scalability**: This refers to how well the software can handle increasing workloads. A scalable architecture ensures that as the system grows (either in terms of data or users), it can expand to meet the demand without sacrificing performance.
- **Maintainability**: A well-architected system is easier to modify and update. It ensures that changes to one part of the system don't cause unintended disruptions elsewhere. This is critical for long-term success as software evolves over time.
- **Flexibility**: A flexible architecture allows for the easy addition of new features or changes to existing functionality without disrupting the entire system. It provides a foundation for growth and adaption.
- **Separation of Concerns**: This is a principle where different parts of the system are responsible for distinct tasks. For example, one module handles data processing, while another manages user interface concerns. This separation makes the system easier to understand and manage.

---

2. Software Architecture Types

A. Model-View-Controller (MVC) Architecture

Definition: The **Model-View-Controller (MVC)** is a popular architectural pattern for designing web and desktop applications. It divides an application into three interconnected components:

1. **Model**: The Model represents the data and business logic of the application. It is responsible for retrieving data from the database, processing it, and passing it to the View. It also responds to changes initiated by the Controller and updates the View.
2. **View**: The View is responsible for presenting data to the user in a user-friendly manner. It displays information from the Model and can also accept user input (e.g., button clicks, text input).
3. **Controller**: The Controller acts as an intermediary between the Model and the View. It takes user input from the View, processes it (e.g., validating the input), and updates the Model accordingly. Once the Model is updated, the Controller may update the View to reflect changes.

When to Use:

- **Web Applications**: MVC is ideal for building web applications where the user interface is separate from the core business logic. Frameworks like Django (Python) and Ruby on Rails use MVC principles.
- **Desktop Applications**: Any application with a user interface (UI) that needs clear separation between data handling (Model) and UI rendering (View) can benefit from MVC.
- **Separation of UI and Logic**: If the application requires a clear division between the logic behind the application (Model) and the way it is presented to the user (View), MVC is a great choice.

MVC Architecture in Python

Statement: In this implementation of the **Model-View-Controller (MVC)** architecture, we will create a simple Python application to manage a list of users. The application will allow adding a user, viewing the list of users, and deleting a user.

- **Model**: This component will handle the data related to users (storing, adding, deleting).
- **View**: This will provide the user interface by displaying the list of users and taking user inputs.
- **Controller**: This will handle the user actions, like adding or deleting users, and update the Model and View accordingly.

Model:

```python
# Model - Handles the data logic
class UserModel:
    def __init__(self):
        self.users = []  # List to store users

    def add_user(self, user_name):
        """Add a user to the list."""
        self.users.append(user_name)

    def delete_user(self, user_name):
        """Delete a user from the list."""
        if user_name in self.users:
            self.users.remove(user_name)
        else:
            print(f"User '{user_name}' not found!")

    def get_all_users(self):
        """Return all users."""
        return self.users
```

View:

```python
# View - Handles the user interface
class UserView:
    def display_users(self, users):
        """Display all users."""
        if not users:
            print("No users available.")
        else:
            print("Users List:")
            for user in users:
                print(f"- {user}")

    def get_user_input(self):
        """Get input from the user."""
        print("\nOptions:")
        print("1. Add User")
        print("2. Delete User")
        print("3. Show Users")
        print("4. Exit")
        return input("Choose an option: ")
```

Controller:

```python
    def run(self):
        """Main method to run the controller."""
        while True:
            choice = self.view.get_user_input()

            if choice == '1':
                user_name = input("Enter user name to add: ")
                self.model.add_user(user_name)
                print(f"User '{user_name}' added!")

            elif choice == '2':
                user_name = input("Enter user name to delete: ")
                self.model.delete_user(user_name)

            elif choice == '3':
                users = self.model.get_all_users()
                self.view.display_users(users)

            elif choice == '4':
                print("Exiting the application.")
                break

            else:
                print("Invalid option. Please try again.")
```

```python
# Instantiate the components
model = UserModel()  # Model
view = UserView()    # View
controller = UserController(model, view)  # Controller

# Run the application
controller.run()
```

Output:

```text
Options:
1. Add User
2. Delete User
3. Show Users
4. Exit
Choose an option: 1
Enter user name to add: John
User 'John' added!

Options:
1. Add User
2. Delete User
3. Show Users
4. Exit
Choose an option: 1
Enter user name to add: Jane
User 'Jane' added!
```

Lab 04

Understanding of Computer Architectures N-Tier

Objective

Will learn how to Implement N-Tier architecture

N-Tier (Layered) Architecture

Definition: The **N-Tier Architecture**, also known as **Layered Architecture**, involves dividing an application into multiple independent layers, each with distinct responsibilities. The most common layers in this model are:

1. **Presentation Layer (UI)**: This layer is responsible for displaying the user interface. It's what the end-user interacts with directly (e.g., the website or application interface). It may include HTML, CSS, JavaScript, and other frontend technologies.
2. **Business Logic Layer**: Also known as the "application layer," this handles the core functionality of the application. It includes the business rules, algorithms, and decisions that define the system's operations. For example, it might handle operations like processing payments or generating reports.
3. **Data Access Layer**: This layer handles all interactions with databases or other data sources. It abstracts the details of data storage and retrieval, ensuring that the business logic does not have to deal with the complexities of data manipulation directly.
4. **(Optional) Additional Layers**: In some systems, there could be additional layers, such as a **Service Layer** for API interactions or a **Security Layer** for user authentication and authorization.

When to Use:

- **Enterprise Applications**: Large-scale systems like customer relationship management (CRM) or enterprise resource planning (ERP) software often use N-Tier architecture. This approach helps keep the application modular and scalable.
- **Modular Development**: If the application needs to be developed in phases or by multiple teams working on separate parts, an N-Tier architecture ensures that the different layers can be developed, tested, and deployed independently.
- **Scalable, Maintainable Systems**: Applications that need to support a large user base and have complex business logic (e.g., online banking, e-commerce platforms) benefit from N-Tier architectures. They can grow as needed, and components can be updated independently of others.
- **Microservices**: Modern applications that use microservices often have an N-Tier architecture where each layer might be a separate microservice. For example, the Data Access Layer could be a microservice dedicated to interacting with a database, while the Business Logic Layer could be a separate microservice focused on processing business operations.

A simple student record management system using N-Tier architecture.

The scenario will include four layers:

1. **Presentation Layer**: For user interaction (UI).
2. **Business Logic Layer**: For managing the core logic and interacting with data.
3. **Data Access Layer**: For managing data storage (simulating database operations with arrays).
4. **Validation Layer** (optional fourth layer): For validating data before it reaches the Business Logic Layer.

In this scenario, we will manage **student records**. The student records will include a name, age, and grade. The user can add, delete, and update student records, and the system will validate inputs (such as age or grade) before they are processed.

Data Layer:

```
# 1. Data Access Layer - Simulating database operations with an in-memory array

class DataAccessLayer:

  def __init__(self):

    self.students = []  # Simulating database with an array


  def add_student(self, student):

    """Add a student record."""

    self.students.append(student)


  def delete_student(self, student_name):

    """Delete a student by name."""

    self.students = [student for student in self.students if student['name'] != student_name]


  def get_all_students(self):

    """Return all student records."""

    return self.students


  def update_student(self, student_name, updated_student):

    """Update student data."""

    for index, student in enumerate(self.students):

      if student['name'] == student_name:
```

```
            self.students[index] = updated_student
            return True
        return False
```

Validation Layer:

```
# 2. Validation Layer - Validates the input data
class ValidationLayer:
    @staticmethod
    def validate_age(age):
        """Validate if the age is a positive integer."""
        return isinstance(age, int) and age > 0

    @staticmethod
    def validate_grade(grade):
        """Validate if the grade is a valid string (A, B, C, etc.)."""
        valid_grades = ['A', 'B', 'C', 'D', 'F']
        return grade in valid_grades

    @staticmethod
    def validate_student_data(name, age, grade):
        """Validate all student data before processing."""
        if not name:
            return False, "Name cannot be empty!"
        if not ValidationLayer.validate_age(age):
            return False, "Age must be a positive integer!"
        if not ValidationLayer.validate_grade(grade):
            return False, "Invalid grade. Valid grades are A, B, C, D, F."
        return True, ""
```

Logic Layer:

```
# 3. Business Logic Layer - Handles the core logic and calls validation and data access layers
class BusinessLogicLayer:
    def __init__(self, data_access_layer, validation_layer):
        self.data_access_layer = data_access_layer
        self.validation_layer = validation_layer

    def add_student(self, name, age, grade):
        """Add a student after validating the data."""
        is_valid, error_message = self.validation_layer.validate_student_data(name, age, grade)
        if is_valid:
            student = {'name': name, 'age': age, 'grade': grade}
            self.data_access_layer.add_student(student)
            return f"Student '{name}' added successfully!"
        else:
            return f"Error: {error_message}"

    def delete_student(self, name):
        """Delete a student from the records."""
        self.data_access_layer.delete_student(name)
        return f"Student '{name}' deleted."

    def update_student(self, name, age, grade):
        """Update student data after validation."""
        is_valid, error_message = self.validation_layer.validate_student_data(name, age, grade)
        if is_valid:
            updated_student = {'name': name, 'age': age, 'grade': grade}
            success = self.data_access_layer.update_student(name, updated_student)
            if success:
                return f"Student '{name}' updated successfully!"
            else:
```

```
            return f"Error: Student '{name}' not found!"
        else:
            return f"Error: {error_message}"

    def get_all_students(self):
        """Retrieve all students."""
        return self.data_access_layer.get_all_students()
```

Presentation Layer:

```
# 4. Presentation Layer - User interface for interacting with the application
class PresentationLayer:
    def __init__(self, business_logic_layer):
        self.business_logic_layer = business_logic_layer

    def display_students(self, students):
        """Display all student records."""
        if not students:
            print("No students found.")
        else:
            print("Student Records:")
            for student in students:
                print(f"Name: {student['name']}, Age: {student['age']}, Grade: {student['grade']}")

    def get_user_input(self):
        """Display options for the user to choose from."""
        print("\nOptions:")
        print("1. Add Student")
        print("2. Delete Student")
        print("3. Update Student")
        print("4. Show All Students")
        print("5. Exit")
        return input("Choose an option: ")
```

```
    def run(self):
        """Main method to interact with the user."""
        while True:
            choice = self.get_user_input()

            if choice == '1':
                name = input("Enter student name: ")
                age = int(input("Enter student age: "))
                grade = input("Enter student grade (A, B, C, D, F): ")
                result = self.business_logic_layer.add_student(name, age, grade)
                print(result)

            elif choice == '2':
                name = input("Enter student name to delete: ")
                result = self.business_logic_layer.delete_student(name)
                print(result)

            elif choice == '3':
                name = input("Enter student name to update: ")
                age = int(input("Enter new student age: "))
                grade = input("Enter new student grade (A, B, C, D, F): ")
                result = self.business_logic_layer.update_student(name, age, grade)
                print(result)

            elif choice == '4':
                students = self.business_logic_layer.get_all_students()
                self.display_students(students)

            elif choice == '5':
                print("Exiting the application.")
                break
```

```
    else:
        print("Invalid option. Please try again.")


# Instantiate the layers
data_access_layer = DataAccessLayer()  # Data Access Layer
validation_layer = ValidationLayer()  # Validation Layer
business_logic_layer = BusinessLogicLayer(data_access_layer, validation_layer)  # Business Logic Layer
presentation_layer = PresentationLayer(business_logic_layer)  # Presentation Layer

# Run the application
presentation_layer.run()
```

Output:

```text
Options:
1. Add Student
2. Delete Student
3. Update Student
4. Show All Students
5. Exit
Choose an option: 1
Enter student name: John
Enter student age: 20
Enter student grade (A, B, C, D, F): A
Student 'John' added successfully!
```

Lab 05

Refactoring In Python

Objective

The objective of this lab is to introduce students to **refactoring**, its importance in software development, and how to apply it in **Python**.

1. What is Refactoring?

Definition:

Refactoring is the process of restructuring existing code to improve its **readability**, **maintainability**, and **efficiency** without altering its external behavior or functionality. It involves making the code cleaner, simpler, and easier to understand while ensuring that the system's functionality remains the same.

The goal of refactoring is to improve the quality of the code over time, making it easier to work with and maintain, while avoiding the introduction of new bugs. Refactoring helps to make the codebase **more sustainable**, **scalable**, and **modular**.

Why is Refactoring Important?

1. **Improves Readability**:
    - Clean code is easier to understand, not just for the person who wrote it but also for other developers working on the codebase in the future. Well-structured code avoids confusion and makes it easier for others to follow the logic.
2. **Enhances Maintainability**:
    - With a well-structured codebase, modifications become easier. When new features need to be added or bugs need to be fixed, clean code is easier to modify without introducing regressions.
3. **Increases Efficiency**:
    - Refactoring can improve the performance of the application. Through removing redundant code, optimizing logic, and improving algorithm efficiency, refactoring can make the code run faster and consume fewer resources.
4. **Reduces Code Duplication**:
    - One of the key aspects of refactoring is removing duplicated code. Redundant logic can often be replaced with reusable functions, reducing the size and complexity of the codebase.
5. **Facilitates Collaboration**:
    - Clean code is essential for collaborative work. When different developers contribute to the same project, well-written and modular code makes it easier to integrate their changes without confusion or conflict.

Real-Life Example:

Consider a banking application that has been running for several years. Over time, developers have added new features, fixed bugs, and made changes. However, the code has become

messy: there are long functions, redundant logic, and unclear variable names. As a result, when a new developer needs to add a feature or fix a bug, they struggle to understand the existing code.

Refactoring would involve simplifying the code, breaking down long functions into smaller ones, eliminating redundant logic, and improving variable names, making it much easier for the developers to work with the system while ensuring that the application still functions as expected.

---

2. Refactoring Legacy Code

What is Legacy Code?

Legacy code refers to **old**, **unoptimized**, or **difficult-to-read code** that is still in use but may not follow modern best practices. This code might have been written using outdated techniques or be poorly structured, making it hard to maintain or extend.

Typical characteristics of legacy code include:

- **Unstructured and lengthy**: Functions that are long and do multiple things, which makes the code harder to read and maintain.
- **Repetitive**: Logic that is repeated throughout the codebase instead of being abstracted into reusable functions or modules.
- **Hard to modify**: Small changes in the code can have unexpected consequences or break existing functionality, making the system fragile.
- **Difficult to understand**: The code might use unclear variable names, lack comments, or use overly complex logic.

Example of Legacy Code (Before Refactoring)

Let’s take an example of poorly written Python code that performs basic arithmetic operations based on a given operation:

```python
def calculate(x, y, operation):
    if operation == "add":
        return x + y
    elif operation == "sub":
        return x - y
    elif operation == "mul":
        return x * y
    elif operation == "div":
        if y == 0:
            return "Error"
        return x / y
    else:
        return "Invalid Operation"

print(calculate(10, 5, "add"))
print(calculate(10, 5, "sub"))
print(calculate(10, 5, "mul"))
print(calculate(10, 5, "div"))
print(calculate(10, 5, "mod"))
```

Problems in This Code:

1. **Too Many If-Else Statements**:
    - The function contains **multiple if-else statements**, each performing a different operation. This makes the function long and difficult to maintain. If a new operation needs to be added, it requires modifying this function, which violates the **Open/Closed Principle** (a principle of object-oriented design that suggests that classes should be open for extension, but closed for modification).
2. **Repetitive Logic**:
    - The same logic (checking if the operation is "add", "sub", "mul", etc.) is written repeatedly for every operation. This leads to code duplication, which makes the code harder to extend and maintain.
3. **Difficult to Extend**:
    - Adding a new operation (such as "mod") requires modifying the calculate function. This creates a tightly coupled design and increases the risk of bugs.

3. Refactoring the Code

After applying **refactoring techniques**, the code becomes simpler, cleaner, and more reusable. Here's the refactored version of the code:

```python
def add(x, y): return x + y
def sub(x, y): return x - y
def mul(x, y): return x * y
def div(x, y): return "Error" if y == 0 else x / y

operations = {
    "add": add,
    "sub": sub,
    "mul": mul,
    "div": div
}

def calculate(x, y, operation):
    return operations.get(operation, lambda x, y: "Invalid Operation")(x, y)

print(calculate(10, 5, "add"))
print(calculate(10, 5, "sub"))
print(calculate(10, 5, "mul"))
print(calculate(10, 5, "div"))
print(calculate(10, 5, "mod"))
```

Improvements in the Refactored Code:

1. **No If-Else Statements**:
2. **Modular and Reusable**:
3. **Easier to Read and Maintain**:
4. **Error Handling**:

Refactoring Techniques.

Refactoring can be performed using different techniques, each focusing on specific aspects of code improvement.

A. Extract Function (Decomposing Large Functions into Smaller Ones)

One of the most common problems in programming is having a **large function** that tries to do multiple things at once. When functions become too long or complex, they become harder to understand and maintain. Refactoring can improve this by breaking the function into smaller, more manageable, and reusable functions.

Before Refactoring:

Consider the following code where a function process_order is responsible for processing an order, calculating the total price, applying a discount, and printing the results. This function does too many things, making it complex and harder to maintain:

```python
def process_order(order):
    print(f"Processing order for {order['customer']}")
    total_price = order['quantity'] * order['price']
    print(f"Total price: {total_price}")
    if total_price > 500:
        print("Applying discount...")
        total_price *= 0.9
    print(f"Final amount to pay: {total_price}")
```

In this example, the discount logic could be separated into its own function for clarity and reusability.

After Refactoring (Extracting a Separate Function):

Here, we've extracted the **discount logic** into a separate function apply_discount. This makes the process_order function more readable and modular.

```python
def apply_discount(price):
    return price * 0.9 if price > 500 else price

def process_order(order):
    print(f"Processing order for {order['customer']}")
    total_price = order['quantity'] * order['price']
    total_price = apply_discount(total_price)
    print(f"Final amount to pay: {total_price}")
```

Benefits:

- The **discount logic** is now reusable.
- The process_order function is cleaner, focusing on high-level tasks, while apply_discount takes care of the discount-specific logic.
- The code is now **modular**, making it easier to maintain and extend.

B. Rename Variables (Use Meaningful Names)

Using **meaningful variable names** is critical for improving code readability. Ambiguous names (like x, y, and z) do not communicate the purpose of the variables, making it difficult to understand what the code does at a glance.

Before Refactoring:

In this example, variables x, y, and z are used for simple multiplication, but their names don't explain what they represent.

```python
x = 10
y = 5
z = x * y  # What does 'z' mean?
```

After Refactoring:

We can replace the ambiguous variable names with more **descriptive names**. In this case, price, quantity, and total_cost provide context for what the variables represent.

```python
price = 10
quantity = 5
total_cost = price * quantity
```

Benefits:

- **Improves readability**: The code is much easier to understand because the names of the variables clearly indicate their purpose.
- **Easier to maintain**: Developers can immediately understand the purpose of a variable and how it relates to the problem at hand.

C. Replace Magic Numbers with Constants

Magic numbers are hardcoded values that are used directly in code without any explanation. These numbers can be confusing because it's not immediately clear what they represent. Replacing magic numbers with **named constants** makes the code more understandable and easier to maintain.

Before Refactoring:

In this example, the number 1.15 is used to calculate the final price, but it is unclear what this number represents (a tax rate, a markup, etc.).

```python
final_price = price * 1.15  # What is 1.15?
```

After Refactoring:

After Refactoring:

We replace the magic number 1.15 with a constant TAX_RATE, which makes it clear that this value represents a tax rate.

```python
TAX_RATE = 1.15
final_price = price * TAX_RATE
```

Benefits:

- **Clarity:** It's now obvious that `1.15` is a **tax rate**.
- **Maintainability:** If the tax rate changes, we only need to update the `TAX_RATE` constant rather than searching for all instances of `1.15` throughout the code.
- **Avoids Duplication:** If the tax rate were used in multiple places, defining it as a constant ensures it's only defined once and can be reused consistently.

Lab 06

Implementing Exception Handling and Robustness Using Python

Objective

Understand **exception handling** and its importance in making code robust. Identify **common types of exceptions** in Python. Improve the **stability of legacy code** by implementing proper exception handling. Apply best practices to make programs **error-resistant and fault-tolerant**.

1. Introduction to Exception Handling

Definition

Exception handling is a technique in programming that allows developers to detect, manage, and recover from errors that occur during runtime. Instead of letting the program crash unexpectedly, exception handling provides a structured way to deal with errors and continue execution smoothly.

Why is Exception Handling Important?

✅ **Prevents Program Crashes:** Ensures the software doesn't abruptly stop due to errors.
✅ **Helps in Debugging:** Provides clear error messages for easier problem identification.
✅ **Enhances User Experience:** Prevents raw system errors from being displayed to users.
✅ **Increases Code Maintainability:** Allows developers to handle exceptions in a structured manner.

2. Common Types of Exceptions in Python

Python provides several built-in exceptions to handle different types of errors. Below are some common ones:

1. `ZeroDivisionError`

Occurs when dividing by zero.

python Copy Edit

```python
x = 10 / 0  # ❌ This will raise ZeroDivisionError
```

2. `ValueError`

Raised when an invalid value is given to a function.

python Copy Edit

```python
num = int("abc")  # ❌ Cannot convert "abc" to an integer
```

3. `TypeError`

Occurs when an operation is performed on incompatible data types.

```python
print("Age: " + 25)  # ❌ Cannot concatenate string with an integer
```

4. `FileNotFoundError`

Occurs when trying to access a file that does not exist.

```python
file = open("missing_file.txt", "r")  # ❌ File does not exist
```

5. `IndexError`

Occurs when accessing an invalid index in a list.

```python
numbers = [1, 2, 3]
print(numbers[5])  # ❌ Index out of range
```

6. `KeyError`

Occurs when accessing a non-existent key in a dictionary.

```python
data = {"name": "Ali"}
print(data["age"])  # ❌ "age" key does not exist
```

3. Implementing Exception Handling in Legacy Code

Legacy code is often written without proper exception handling, making it prone to errors and crashes. Let's analyze an example of poorly written code and improve it using exception handling.

Legacy Code (Before Exception Handling)

The following program calculates the average of numbers entered by the user. However, it contains multiple issues:

```
def calculate_average():
    numbers = []
    while True:
        num = input("Enter a number (or 'q' to quit): ")
        if num == "q":
            break
        numbers.append(int(num))  # ❌ No error handling for invalid input
    return sum(numbers) / len(numbers)  # ❌ May cause ZeroDivisionError if no numbers are entered
print("Average:", calculate_average())
```

Problems in This Code

❌ Crashes if the user enters invalid input (e.g., entering "abc" instead of a number).
❌ Crashes with ZeroDivisionError if no numbers are entered.
❌ No mechanism to handle unexpected errors.

4. Refactored Code (After Exception Handling)

We will now modify the code to handle errors gracefully using try-except blocks.

```
def calculate_average():
    numbers = []
    while True:
        try:
            num = input("Enter a number (or 'q' to quit): ")
            if num.lower() == "q":
                break
            numbers.append(float(num))  # ✅ Handles non-integer values like 5.5
        except ValueError:
            print("❌ Invalid input! Please enter a valid number.")
```

```
    try:
        return sum(numbers) / len(numbers)  # ✅ Handles division by zero
    except ZeroDivisionError:
        return "❌ No numbers entered! Cannot calculate average."

print("Average:", calculate_average())
```

Improvements in the Refactored Code

✓ **Handles Invalid Inputs:** Prevents crashes when users enter non-numeric values.
✓ **Prevents Division by Zero:** Gracefully handles cases where no numbers are entered.
✓ **Improves User Experience:** Provides clear error messages instead of system errors.
✓ **More Robust and Maintainable:** Future improvements can be added easily.

Exception Handling Techniques in Python

Introduction

Exception handling is a fundamental concept in programming that allows developers to detect, handle, and recover from runtime errors efficiently. Python provides various techniques to manage exceptions, making the code more robust, readable, and user-friendly.

In this section, we will explore different exception handling techniques along with **definitions, examples, and best practices** to write better code.

1. Using try-except Blocks

Definition

A try-except block is used to catch and handle exceptions instead of allowing the program to crash. The try block contains the code that may cause an error, and the except block handles the exception gracefully.

Example: Handling Division Errors

```
try:
    result = 10 / 0  # ❌ ZeroDivisionError
except ZeroDivisionError:
    print("❌ Cannot divide by zero! Please enter a valid denominator.")
```

Explanation

- The division operation 10 / 0 would normally cause a ZeroDivisionError.
- The except block catches this error and displays a user-friendly message instead of terminating the program.

2. Using Multiple except Blocks

Definition

Sometimes, different types of errors can occur in a single block of code. By using **multiple except blocks**, we can handle each type of exception separately, providing appropriate error messages for each case.

Example: Handling Multiple Errors

```
try:
    num = int(input("Enter a number: "))  # May cause ValueError
    result = 10 / num  # May cause ZeroDivisionError
except ValueError:
    print("❌ Invalid input! Please enter a numeric value.")
except ZeroDivisionError:
    print("❌ Cannot divide by zero! Enter a non-zero number.")
```

Explanation

- If the user enters a non-numeric value (e.g., "abc"), a ValueError occurs.
- If the user enters 0, a ZeroDivisionError occurs.
- Each error is handled separately, providing **clear and relevant** feedback to the user.

3. Using finally Block

Definition

The finally block is used to **execute code that must run regardless of whether an exception occurs or not**. It is commonly used for **clean-up operations**, such as closing files, releasing resources, or disconnecting from a database.

Example: Ensuring File Closure

```
try:
    file = open("data.txt", "r")
    content = file.read()
except FileNotFoundError:
    print("❌ File not found!")
finally:
    file.close()  # Ensures file is closed
```

Explanation

- If the file "data.txt" does not exist, a FileNotFoundError occurs.

- Regardless of whether an error occurs or not, the finally block ensures that the file is closed properly.
- This prevents **resource leaks** and ensures the program runs efficiently.

4. Using Custom Exceptions

Definition

Python allows developers to define **custom exception classes** to handle specific application errors. This improves **code readability and maintainability** by making errors more meaningful and context-specific.

Example: Custom Exception for Invalid Age

```
class InvalidAgeError(Exception):
    """Custom exception for invalid age input."""
    pass


def check_age(age):
    if age < 0:
        raise InvalidAgeError("❌ Age cannot be negative!")


try:
    check_age(-5)
except InvalidAgeError as e:
    print(e)
```

Explanation

- We create a **custom exception** InvalidAgeError that inherits from Python's built-in Exception class.
- The function check_age() raises this custom exception if the age is negative.
- The except block catches the custom exception and displays the appropriate message.

Lab 07

Software Testing in python

Objective

Understand the basics of **software testing** and its importance. Learn about **unit testing** and **PyTest** in Python. Write and run **test cases** using **PyTest**. Perform assertions to verify expected outputs. Work on a hands-on task to **test a simple Python application**.

Introduction to Software Testing

What is Software Testing?

Software testing is the **process of evaluating** a software application to ensure it functions **correctly, meets user requirements, and is free from defects**. It helps identify bugs, improves software reliability, and enhances performance.

Why is Software Testing Important?

✅ Ensures software works as expected before deployment.
✅ Improves the quality and reliability of the application.
✅ Identifies defects and security vulnerabilities.
✅ Saves time and cost by detecting errors early in development.
✅ Enhances user satisfaction by providing a stable and functional product.

Types of Software Testing

Software testing is broadly categorized into **manual testing** and **automated testing**. Within these, there are multiple testing levels and techniques.

Unit Testing

- Tests **individual components (functions, methods, or modules)** of a program.
- Ensures that each function works correctly in **isolation**.
- Example: Testing a function that adds two numbers.

Integration Testing

- Checks how **different modules or components interact** with each other.
- Ensures smooth communication between **databases, APIs, or other software components**.
- Example: Testing if the user authentication system correctly connects to the database.

System Testing

- Tests the **entire application** as a whole.
- Ensures the software meets **functional and non-functional requirements**.
- Example: Testing a complete **e-commerce website** to check if users can add products to the cart and make payments.

Regression Testing

- Ensures that **new code changes** do not break existing functionality.
- Used to verify that bug fixes, updates, or new features **don't introduce new issues**.
- Example: After adding a new feature to a social media app, regression testing ensures that old features like messaging and notifications still work correctly.

Functional vs. Non-Functional Testing

- **Functional Testing**: Tests if the application works **as intended** (e.g., unit, integration, system testing).
- **Non-Functional Testing**: Evaluates **performance, security, usability, and reliability**.

PyTest: A Powerful Testing Framework in Python

What is PyTest?

PyTest is a widely used Python testing framework that simplifies writing and executing test cases. It is designed to make testing easier, more readable, and efficient. Unlike Python's built-in unittest module, PyTest allows writing tests using simple functions and provides **automatic test discovery, powerful assertions, and detailed failure reports**.

PyTest is used for **unit testing, functional testing, and even API testing**, making it a preferred choice for developers working on large-scale projects.

Why Use PyTest?

PyTest is favored by developers because of its several advantages:

- **Simple and Readable Syntax** – No need to create test classes; just use plain functions.
- **Automatic Test Discovery** – Finds test cases automatically without requiring manual setup.
- **Powerful Assertions** – Uses Python's built-in assert statement for validation.
- **Fixture Support** – Allows efficient management of setup and cleanup for tests.
- **Parameterized Testing** – Tests functions with multiple inputs using @pytest.mark.parametrize.
- **Detailed Error Messages** – Provides better debugging information when tests fail.
- **CI/CD Integration** – Easily integrates with Jenkins, GitHub Actions, and other CI/CD tools.

Core Features of PyTest

1. Writing Basic Test Cases

PyTest does not require complex class-based structures. You can write simple test functions using the assert statement.

Example: Basic PyTest Test Case

```
# test_math.py


def multiply(a, b):
    return a * b


def test_multiply():
    assert multiply(2, 3) == 6
    assert multiply(0, 5) == 0
```

Expected Output:

```
==================== test session starts ====================
collected 1 item


test_math.py .                                   [100%]


===================== 1 passed in 0.12s =====================
```

---

2. Running PyTest

To execute tests in Google Colab, use:

```
!pytest test_math.py
```

This command will run all test cases inside the file and display the results.

To run all tests in the project directory:

```
!pytest
```

---

3. Handling Exceptions Using pytest.raises

PyTest provides pytest.raises to check if a function raises an expected exception.

Example: Testing for ZeroDivisionError

```
import pytest
```

```
def divide(a, b):
    return a / b

def test_divide():
    with pytest.raises(ZeroDivisionError):
        divide(5, 0)
```

Expected Output:

```
==================== test session starts ====================
collected 1 item

test_math.py .                          [100%]

====================== 1 passed in 0.10s =======================
```

If the function does not raise ZeroDivisionError, the test will fail.

---

4. Using Fixtures in PyTest

Fixtures help in setting up reusable test data and cleaning up resources after the test completes.

Example: Using a Fixture

```
import pytest

@pytest.fixture
def sample_data():
    return [10, 20, 30]

def test_sum(sample_data):
    assert sum(sample_data) == 60
```

Expected Output:

```
==================== test session starts ====================
collected 1 item

test_math.py .                          [100%]
```

```
====================== 1 passed in 0.08s =======================
```

---

5. Running Tests with Multiple Inputs Using @pytest.mark.parametrize

PyTest supports parameterized testing, allowing a function to be tested with different inputs.

Example: Parameterized Testing

```
import pytest

@pytest.mark.parametrize("a, b, expected", [
    (2, 3, 5),
    (5, 5, 10),
    (1, 1, 2)
])
def test_add(a, b, expected):
    assert add(a, b) == expected
```

Expected Output:

```
==================== test session starts ====================
collected 3 items

test_math.py ...                          [100%]

====================== 3 passed in 0.15s =======================
```

PyTest automatically executes the test three times, each with different input values.

---

6. Example of a Failing Test Case

Sometimes, test cases fail due to incorrect function logic or unexpected outputs.

Example: A Failing Test Case

```
def subtract(a, b):
    return a - b

def test_subtract():
    assert subtract(10, 5) == 6  # Incorrect expected value
```

Expected Output (Failure Message):

```
==================== test session starts ====================
collected 1 item

test_math.py F                                   [100%]

===================== FAILURES =====================
___________ test_subtract ___________

  def test_subtract():
>     assert subtract(10, 5) == 6
E     assert 5 == 6
E     + where 5 = subtract(10, 5)

test_math.py:6: AssertionError
```

In this case, the actual output 5 does not match the expected output 6, causing an **AssertionError**.

Lab 08

Making an Executable .exe file

Objective

Understand the **need for executable files (.exe)**.Convert a basic Python script into a **standalone application** (.exe). Use **PyInstaller** to generate an .exe file. Run and distribute Python applications **without requiring Python installation**.

Creating Executable (.exe) Files from Python Scripts

1. Introduction to Executable Files (.exe)

What is an Executable File?

An executable file (.exe) is a **standalone application** that runs on **Windows** without requiring Python to be installed. It contains all the necessary code, dependencies, and libraries needed to run a Python script as an independent program.

Why Convert a Python Script to an .exe File?

Creating an **executable file** has several advantages:

- **Easier to Share** – Users don't need Python installed to run the program.
- **Protects Source Code** – The script is compiled, making it harder to access the original code.
- **GUI Applications** – Allows creating applications without requiring the command line.
- **Professional Look** – Makes the program appear as a real software product.

---

2. Installing PyInstaller

What is PyInstaller?

PyInstaller is a Python package that converts Python scripts into **Windows executable files (.exe)**. It bundles all dependencies, making the program self-contained.

How to Install PyInstaller?

To install PyInstaller, open a **Command Prompt (cmd)** or **terminal** and run:

bash

```
pip install pyinstaller
```

Once installed, you can check if PyInstaller is available using:

bash

pyinstaller --version

---

3. Writing a Basic Python Program

Let's create a **simple Python script** to add two numbers.

Step 1: Create a Python File (addition.py)

Open a text editor or an IDE (such as VS Code, PyCharm, or Notepad++) and write the following code:

python

```
def add_numbers():
    num1 = float(input("Enter first number: "))
    num2 = float(input("Enter second number: "))
    result = num1 + num2
    print(f"Sum: {result}")

if __name__ == "__main__":
    add_numbers()
```

Save the file as addition.py.

4. Converting the Python Script to an .exe File

Step 2: Open Command Prompt and Navigate to the Script's Location

Use the cd command to go to the folder where addition.py is saved:

bash

```
cd path\to\your\script
```

For example, if your script is in C:\Users\Rayan\Documents, run:

bash

```
cd C:\Users\Rayan\Documents
```

Step 3: Run PyInstaller to Create the Executable File

Run the following command in the terminal:

bash

```
pyinstaller --onefile addition.py
```

Explanation:

- --onefile → Creates a **single** .exe file instead of multiple files.
- addition.py → The script to convert into an executable.

Running PyInstaller in Google Colab

Since **Google Colab does not support direct execution of .exe files**, follow these steps to **create the .exe in Colab** and **download it to your computer**:

Step 1: Install PyInstaller in Colab

```
!pip install pyinstaller
```

Step 2: Write the Python Script in Colab

```
%%writefile addition.py
def add_numbers():
    num1 = float(input("Enter first number: "))
    num2 = float(input("Enter second number: "))
    result = num1 + num2
    print(f"Sum: {result}")

if __name__ == "__main__":
    add_numbers()
```

Step 3: Run PyInstaller in Colab

```
!pyinstaller --onefile addition.py
```

Step 4: Download the Executable File from Colab

```
from google.colab import files
files.download("dist/addition.exe")
```

This will allow you to **download addition.exe to your computer** and run it locally.

Creating a GUI-Based Calculator (.exe) in Google Colab Using Tkinter

Since Google Colab does not support GUI applications natively, we will follow a step-by-step process to write the code, convert it into an .exe file using PyInstaller, and download it to run on Windows.

Step 1: Install PyInstaller in Google Colab

Google Colab does not have PyInstaller installed by default, so it needs to be installed first.

```
!pip install pyinstaller
```

Step 2: Write the GUI Calculator Script in Colab

Use the %%writefile command to create and save the Python script in Colab.

```
%%writefile gui_calculator.py
import tkinter as tk

def calculate_sum():
    try:
        num1 = float(entry1.get())
        num2 = float(entry2.get())
        result.set(num1 + num2)
    except ValueError:
        result.set("Invalid Input")

# Creating GUI Window
root = tk.Tk()
root.title("Simple Calculator")

# Entry fields for numbers
entry1 = tk.Entry(root)
entry1.pack()

entry2 = tk.Entry(root)
entry2.pack()
```

```
# Display result
result = tk.StringVar()
label = tk.Label(root, textvariable=result)
label.pack()

# Button to calculate sum
button = tk.Button(root, text="Add", command=calculate_sum)
button.pack()

# Run the GUI loop
root.mainloop()
```

This script creates a simple GUI-based calculator where users can enter two numbers, press the "Add" button, and see the sum displayed. If an invalid input is entered, it will display an error message instead of crashing.

Step 3: Convert Python Script to an Executable (.exe) Using PyInstaller

Now, we will convert the script into an .exe file using PyInstaller in Colab.

```
!pyinstaller --onefile --windowed gui_calculator.py
```

Explanation:

- --onefile creates a single .exe file instead of multiple files.
- --windowed hides the command prompt when running the GUI.

Once the command executes successfully, PyInstaller will create a "dist" folder containing gui_calculator.exe.

Step 4: Download the .exe File from Colab

Since we are using Colab, we need to download the generated .exe file to run it on Windows.

```
from google.colab import files
files.download("dist/gui_calculator.exe")
```

This will prompt a download of the gui_calculator.exe file. You can then double-click on it in Windows to open the calculator without needing Python installed.

Lab 9

Introduction to GitHub and Repository Management

Objective
This lab will introduce students to GitHub, its importance in software development, and the process of setting up and managing repositories.
Concepts Covered

1. **What is GitHub?**
   GitHub is a cloud-based platform that enables developers to collaborate on projects, manage code versions, and track changes efficiently. It is built on Git, a distributed version control system, allowing multiple contributors to work on the same project without conflicts.
2. **Why Use GitHub?**
   - Centralized storage for code and projects
   - Tracks changes and maintains a history of modifications
   - Facilitates collaboration among developers
   - Automates deployment and integrates with CI/CD pipelines
3. **Key GitHub Features**
   - **Repositories**: A storage location for project files, including code and documentation
   - **Commits**: Individual changes or updates to a project
   - **Branches**: Different versions of a project, allowing parallel development
   - **Pull Requests**: A way to propose and review changes before merging
   - **Issues**: Used for tracking bugs, feature requests, and project discussions
   - **Releases**: Versioned snapshots of a project for easy distribution

---

Step-by-Step Guide: Setting Up GitHub and Creating a Repository
Step 1: Sign Up for GitHub

1. Open a web browser and go to GitHub.
2. Click "Sign Up" and fill in the required details (username, email, and password).
3. Complete the verification process and log in to your new GitHub account.

Step 2: Create a New Repository

1. Click the "+" sign in the top-right corner and select "New Repository."
2. Enter a repository name (e.g., MyFirstRepo).
3. Choose the visibility:
   - **Public**: Anyone can see your repository.
   - **Private**: Only you and collaborators can access it.
4. Check “Add a README file” to include an introductory file.
5. Click “Create Repository.”

Step 3: Install Git on Your System

1. Download and install Git from Git's official website.
2. Open a terminal or command prompt and verify the installation:
   git –version

Step 4: Configure Git
```
git config --global user.name "Your Name"
git config --global user.email "your-email@example.com"
```

Step 5: Clone a Repository
To download a repository to your local system:
git clone https://github.com/your-username/MyFirstRepo.git
This creates a local copy of your GitHub repository.

Lab 10

Github Version Control

Objective

This lab will teach students how to integrate GitHub with Thonny, manage versions of their code, and track changes efficiently.

Concepts Covered

1. **What is Version Control?**
   Version control is a system that records changes to a file or set of files over time, allowing users to revert to previous versions and track modifications.
2. **Types of Version Control Systems (VCS)**
   - **Local VCS**: Stores changes on a single machine.
   - **Centralized VCS (CVCS)**: Uses a central server to store all versions.
   - **Distributed VCS (DVCS)**: Each user has a full copy of the repository (e.g., Git).
3. **Benefits of Using Version Control**
   - Prevents loss of work by maintaining a history of changes.
   - Enables collaboration by merging changes from multiple contributors.
   - Allows experimentation with new features without affecting the main code.

Step-by-Step Guide: Using GitHub with Thonny

Step 1: Install Thonny and Enable Git

1. Download and install Thonny.
2. Open Thonny and go to **Tools → Options → General**.
3. Ensure that "Use Git for version control" is enabled.

Step 2: Clone a GitHub Repository in Thonny

1. In Thonny, go to **Tools → Open Repository**.
2. Enter the GitHub repository URL and choose a local directory.
3. Click "Clone" to download the repository.

Step 3: Create and Modify a Python File

1. In Thonny, create a new Python file (calculator.py):

python

CopyEdit

def add(a, b):

```
    return a + b


print("Sum:", add(5, 3))
```

2. Save the file inside the cloned repository folder.

Step 4: Commit Changes to GitHub

1. Open Thonny's **Version Control** panel.
2. Click "Stage" to add changes to the commit.
3. Write a commit message (e.g., "Added a calculator function").
4. Click "Commit" to save changes locally.

Step 5: Push Changes to GitHub

1. Click "Push" to upload changes to GitHub.
2. Enter your GitHub credentials when prompted.
3. Verify that the changes appear in the GitHub repository.

Step 6: Pull Updates from GitHub

If another user makes changes, you can download the latest version:

1. Click "Pull" in Thonny.
2. The latest changes will be merged into your local repository.

Lab 11

Personal and Peer Reviews

Objective:

1. Understand the importance of **personal and peer reviews** in software development.
2. Learn how to **conduct personal code reviews** before committing changes.
3. Practice **peer reviewing code using GitHub's Pull Requests and Issues**.
4. Apply best practices for **reviewing code and providing constructive feedback**.

---

1. Introduction to Code Reviews

What is a Code Review?

A code review is a process where developers examine each other's code to ensure **quality, security, and maintainability** before merging changes into the main project.

Why are Code Reviews Important?

- **Catch bugs early** before they reach production.
- Improve **code readability and maintainability**.
- Ensure **coding standards and best practices** are followed.
- Facilitate **knowledge sharing** among team members.

Types of Code Reviews

1. **Personal Review** – Reviewing your own code before sharing it.
2. **Pair Programming** – Writing code together in real-time.
3. **Formal Peer Review** – Structured reviews using **Pull Requests and Comments**.
4. **Ad-hoc Review** – Informal discussion on code improvements.

---

2. Setting Up a GitHub Repository for Code Reviews

Before starting peer reviews, we need to set up a repository.

Step 1: Create a Repository on GitHub

1. **Go to GitHub** and log in.
2. Click on **New Repository**.
3. Enter a **repository name** (e.g., python-review).
4. Select **Public or Private**.
5. Click **Create Repository**.

Step 2: Clone the Repository in Thonny

1. Open **Thonny**.
2. Go to **Tools → Manage Plug-ins** and install **Git** if not already installed.
3. Open **View → Git** to enable Git integration.
4. Click **Clone Repository** and paste the GitHub URL.
5. Choose a local folder and click **Clone**.

---

3. Conducting a Personal Code Review

Before submitting code for peer review, developers should review their own work.

Checklist for Personal Code Review

✓ Is the code **clear and well-documented**?
✓ Does the code **follow PEP8 style guidelines**?
✓ Have you tested for **edge cases and errors**?
✓ Are all **variables and functions meaningfully named**?
✓ Have you removed **unnecessary print statements**?

Example: Code Before Personal Review

```
def sum(a, b):
  return a+b
print(sum(5,3))
```

Issues:

- Function name is unclear (sum is a built-in function).
- No proper spacing or comments.

Code After Personal Review

```
def add_numbers(num1, num2):
  """Returns the sum of two numbers"""
  return num1 + num2

# Example usage
print(add_numbers(5, 3))
```

Improvements:

- **Meaningful function name** (add_numbers).
- **Added a docstring** for documentation.
- **Used spaces around + operator** (PEP8 formatting).

---

4. Submitting Code for Peer Review on GitHub

After reviewing your own code, you submit it for peer review.

Step 1: Create a New Branch

1. Open **Thonny**.
2. Click on **Git → Branches → New Branch**.
3. Name the branch (e.g., feature-addition).
4. Click **Create Branch**.

Step 2: Make Changes and Commit

1. Edit your code.
2. Go to **Git → Commit Changes**.
3. Add a **commit message** (e.g., "Refactored addition function").
4. Click **Commit**.

Step 3: Push the Branch to GitHub

1. Click **Git → Push**.
2. Select the branch (feature-addition).
3. Click **Push**.

---

5. Conducting a Peer Review on GitHub

Now, your teammates can review your code.

Step 1: Open a Pull Request

1. Go to **GitHub → Your Repository**.
2. Click on **Pull Requests → New Pull Request**.
3. Select the feature-addition branch.
4. Click **Create Pull Request**.
5. Add a **title and description** explaining the changes.

Step 2: Reviewing Code as a Peer

1. Click on the **Pull Request** in GitHub.
2. Select **Files Changed** to see the code.
3. Use **Add Comment** to suggest improvements.
4. Click **Approve, Request Changes, or Comment**.

Step 3: Merging Changes

1. Once approved, click **Merge Pull Request**.
2. Click **Delete Branch** to clean up.

Lab 12

Open Ended to make Project

Objective
Students will apply all the concepts they have learned in previous labs to develop a complete software project. Each student will work on a unique project to ensure originality. The project will involve:

- **Planning & Design** (SDLC, diagrams)
- **Software Architecture** (MVC & N-Tier)
- **Code Quality & Maintenance** (Refactoring, Exception Handling, Unit Testing)
- **Deployment** (.exe file creation)
- **Version Control & Collaboration** (GitHub repositories, releases, peer reviews)

Question:
You have been assigned the task of developing a **Personal Expense Tracker** application that allows users to **add, view, delete, and edit expenses** using a **Tkinter-based GUI**. The application should store expenses in a **list (array)** and allow users to **save and load** their data from a text file. To ensure reliability, you must handle **invalid inputs and errors** through exception handling. Additionally, the application should be tested using **PyTest**, converted into an **executable (.exe) file**, and finally, uploaded to **GitHub with proper version control and peer reviews**.
To complete this project, you must first **plan** the development process by creating a **Gantt chart** and defining the **software architecture** through **use case diagrams, sequence diagrams, and component diagrams**. Once the planning phase is complete, you will implement the application, ensuring that it follows the **MVC architecture** and includes proper **error handling and unit tests**. After successful development and testing, you will create a **GitHub repository**, push your code, and generate a **GitHub release** for version management. Finally, you will conduct **peer reviews** to evaluate your classmates' projects and provide constructive feedback.